Papelería Matrix: qué se hizo y qué sigue

Lo que quedó en `main` hoy, cómo entenderlo y qué le toca hacer a cada uno. Para Adrián.

En una frase

Hoy quedó en `main` un sistema que ya funciona de punta a punta: la **base de datos** con los scripts corregidos, una **API** en Spring Boot que entrega los servicios, la papelería y la ropa, una **página pública** que usa esa API y un **CI en GitHub** que prueba todo antes de aceptar un cambio. Todavía faltan el **login real** y el **panel interno** de la administradora y los operadores.

Este paquete tiene tres documentos: este resumen, el **plan de pruebas** y la **estructura del proyecto** actualizada. Los dos últimos ya están redactados como borrador, para que los revises y los hagas tuyos.

Lo que se hizo: los 9 pull requests

Un pull request (PR) es un paquete de cambios que se revisa antes de entrar a `main`. Los nueve están mergeados.

#	RAMA	QUÉ HACE, EN SIMPLE
1	`Refactor/estructura-proyecto`	Ordena el repositorio en `database`, `backend`, `frontend` y las carpetas de documentación. Todo en minúscula para que no falle entre sistemas.
2	`Fix/scripts-sql`	Corrige los 30 scripts SQL: tablas en minúscula, `utf8mb4` para tildes y eñes, la tabla de estadísticas y el orden de las relaciones. Probados en MariaDB 10.4, la de XAMPP: 15 tablas y 14 relaciones sin errores.
3	`Feat/vista-login`	Trae al repositorio la vista de login que había hecho Jaber.
4	`Feat/spring-boot-base`	Backend base con Spring Boot 4.1 y Java 21: ruta `/api/estado`, documentación Swagger, `Dockerfile` y `render.yaml` para publicar en Render.
5	`Feat/api-catalogo`	La API de verdad: servicios (alta, consulta, edición y baja, con clave `X-API-Key`), productos y ropa (consulta). Errores claros en JSON, datos de ejemplo para el modo demo y una colección de Postman con 15 peticiones.
6	`Feat/ci-github-actions`	El CI: en cada PR compila, corre las pruebas, construye la imagen de Docker y comprueba que la API arranca. Render solo despliega si todo pasa.
7	`Docs/flujo-de-trabajo`	El archivo `CONTRIBUTING.md`, con el paso a paso para trabajar con ramas y PR. Fue el PR de prueba para ver el flujo.
8	`Fix/rutas-inexistentes`	Abrir la raíz o una ruta que no existe daba un error 500. Ahora responde 404 o 405 con un mensaje claro, y la raíz lleva a Swagger.

9	`Feat/frontend-publico`	El sitio público conectado a la API: catálogos con buscador, buscador general, menú de celular, términos y aviso de privacidad, y el login con la marca. Incluye CORS en el backend.

Hay 24 pruebas automáticas en el backend y todas pasan en el CI.

Cómo está armado

Frontend
HTML, CSS y JavaScript. Es la página que ve el cliente.
`localhost:5500`

→

API (backend)
Spring Boot y Java 21. Recibe peticiones y responde JSON.
`localhost:8080`

→

Base de datos
MySQL de XAMPP en modo local. H2 en memoria en modo demo.

La API corre en dos modos, que se eligen con la variable `SPRING_PROFILES_ACTIVE`:

- **local:** usa el MySQL de XAMPP con las tablas reales. Es el sistema de verdad.
- **demo:** usa una base H2 en memoria con datos de ejemplo. Es lo que se publica en Render y lo que usan las pruebas. Los datos se pierden al reiniciar.

Las capas del backend, y dónde quedó el DAO

Cada petición pasa por las mismas capas. `Servicio` es el ejemplo completo y el modelo a copiar.

CAPA	QUÉ HACE	EJEMPLO
Controller	Recibe la petición HTTP (`GET /api/servicios`) y devuelve la respuesta.	`controller/ServicioController`
Service	Las reglas del negocio: código repetido es error 409, no existe es error 404.	`service/ServicioService`
Repository	Habla con la base de datos. Es el reemplazo del DAO: antes se escribía el JDBC a mano, ahora Spring lo genera y solo se declara qué buscar.	`repository/ServicioRepository`
Entity (`model`)	Una tabla de la base vista como clase de Java.	`model/Servicio`
DTO	Lo que entra y sale en JSON. Así el público no ve datos internos, como el precio de compra.	`dto/ServicioRequest` , `ServicioResponse`

> **Idea clave para los DAO.** Las clases de `model` y `dao` con JDBC siguen en el repositorio para las 12 tablas que faltan. La lógica que ya está escrita no se tira: se mueve. Las validaciones de los *setters* pasan a anotaciones (`@NotBlank` , `@Size` , `@Positive`) y las consultas de `UsuarioDAO` pasan a métodos del repositorio.

Mapa de carpetas

CARPETA	QUÉ HAY
`database/scripts`	Scripts SQL en orden: 01 crear base, 02 crear tablas, 03 relaciones.
`backend`	La API. El código está en `src/main/java/com/papeleriamatrix` y las pruebas en `src/test` .
`frontend`	La página: `index.html` , `views/login.html` , `css` , `js` y `assets` .

`postman`	La colección de pruebas de la API y sus entornos Local y Render.
`.github/workflows`	El CI (`ci.yml`).
`01-requisitos` a `08-contexto`	La documentación del proyecto.

Cómo probarlo en tu computador

1. **Actualiza el repositorio:** `git switch main` y luego `git pull` .
2. **Base de datos, solo para el modo local:** enciende MySQL en XAMPP y ejecuta los scripts de `database/scripts` en orden: carpetas 01, 02 y 03.
3. **Backend:** dentro de la carpeta `backend` , en modo demo, que no necesita XAMPP:

   ```
   mvn spring-boot:run "-Dspring-boot.run.profiles=demo"
   ```

   Las comillas son necesarias en PowerShell. Cuando salga `Started PapeleriaMatrixApplication` , abre `http://localhost:8080` : lleva a Swagger, donde se prueban todas las rutas. Se apaga con `Ctrl+C` . En modo local, el mismo comando sin el perfil demo.
4. **Frontend:** desde la raíz del repositorio, `jwebserver -d frontend -p 5500` , que viene con el JDK 21, y abre `http://localhost:5500` .
5. **Postman:** importa los 3 archivos de la carpeta `postman` , elige el entorno Local y pulsa Run.
6. **Pruebas automáticas:** dentro de `backend` , `mvn verify` . Deben pasar las 24.

La clave de la API en local es `clave-de-prueba` . Se envía en la cabecera `X-API-Key` para crear, editar o borrar.

Cómo trabajamos

1. Rama desde `main` , con nombre `Feat/…` o `Fix/…` → **2. Commits** pequeños, con prefijo `feat:` , `fix:` , `docs:` → **3. Pull request** hacia `main` , con qué cambia y cómo probarlo → **4. CI** 2 revisiones en verde: Backend y Docker → **5. Merge** otra persona lo revisa y se mergea

Nadie sube directo a `main` . El detalle está en `CONTRIBUTING.md` . Nunca se suben contraseñas ni claves al repositorio.

Estado frente a la documentación

REQUISITOS	QUÉ HAY	ESTADO
RF-01: consultar el catálogo	Página pública y API con buscadores.	• **Hecho**
RF-03 a RF-06: servicios	API de alta, edición, baja y consulta. Falta la pantalla y el control por rol.	◐ **Parcial**
RF-07 a RF-10: productos de papelería	Solo consulta. Falta alta, edición y baja.	◐ **Parcial**
RF-20 a RF-23: ropa	Solo consulta. Falta alta, edición y baja.	◐ **Parcial**

RF-11 a RF-14: información del negocio	Los horarios están escritos en el HTML. Sin API.	○ **Pendiente**
RF-15 a RF-18, RF-28, RF-29: operadores y claves	Solo existen las clases antiguas.	○ **Pendiente**
RF-24 a RF-27: facturas de servicios	Nada todavía.	○ **Pendiente**
RF-30 a RF-37: ventas de papelería y de ropa	Nada todavía.	○ **Pendiente**
RF-19: ingresos, estadísticas y reportes	Nada todavía.	○ **Pendiente**
RF-38, RNF-01, RNF-18: login, contraseñas con hash, sesión de 15 minutos	Solo la pantalla de login, sin acceso real.	○ **Pendiente**
RNF-12, RNF-16, RNF-05: responsive, términos, errores amigables	Hechos en la página y en la API.	● **Hecho**
RNF-02, RNF-09: accesibilidad y 2,5 segundos	Accesibilidad básica sin auditoría. El tiempo no se midió.	◐ **Parcial**

Las tareas de Adrián

Hay tareas de código, para practicar sobre las 12 tablas que todavía tienen el DAO antiguo, y tareas de documentación. Cada tarea de código es una rama y un PR, y el modelo a copiar es `Servicio`.

El molde, paso a paso

1. **Rama** desde `main`: `git switch -c Feat/nombre-de-la-tarea`.
2. **Entity:** una clase con `@Entity` y `@Table(name = "...")`, y un `@Column` por cada columna del script SQL. Mira `model/Servicio.java`.
3. **Repository:** una interfaz que extiende `JpaRepository`. Mira `repository/ServicioRepository.java`. Este paso reemplaza al DAO.
4. **DTO:** un `…Request` con las validaciones y un `…Response` con lo que se devuelve.
5. **Service:** las reglas del negocio y los errores 404 y 409. Mira `ServicioService.java`.
6. **Controller:** las rutas REST, con `@Operation` para que salgan en Swagger. Las escrituras llevan `@SecurityRequirement(name = "claveApi")`.
7. **Pruebas:** copia `ServicioControllerTest` y cambia los datos. `mvn verify` debe pasar.
8. **Extras:** datos de ejemplo en `demo/datos-demo.sql`, peticiones en `postman` y el PR con sus dos revisiones en verde.

Tareas de código

TAREA	QUÉ HAY QUE HACER	REQUISITOS	NIVEL
T1 Productos: alta, edición y baja	Agregar `POST`, `PUT` y `DELETE` a `/api/productos`. El `ProductoRequest` sí lleva precio y lugar de compra; la respuesta pública no.	RF-07, 08, 09 CU-06, 07, 08	1 de 4 Calentamiento
T2 Ropa: alta, edición y baja	Lo mismo que T1, sobre `/api/ropa`.	RF-20, 21, 22 CU-36, 37, 38	1 de 4 Calentamiento

T3 Usuario y Administrador	Pasar `Usuario` y `Administrador` a entidad y repositorio, aprovechando lo que ya hay en `UsuarioDAO` y `AdministradorDAO`. La contraseña se guarda con hash BCrypt, nunca en texto plano.	RNF-01	2 de 4
T4 Información del negocio	Entidad, repositorio, servicio y controlador. Consultar es público, escribir pide clave. Después la página leerá los horarios de aquí en vez del HTML.	RF-11 a 14 CU-11 a 15	2 de 4 Depende de T3
T5 Operadores y claves de permiso	Contratar, actualizar, despedir, listar y consultar operadores, y crear o cambiar sus claves. Parte de `Operador_de_cibercafeDAO`, que ya tiene código. `Clave_de_permisoDAO` está vacío y se escribe desde cero.	RF-15 a 18, 28, 29 CU-16 a 23	3 de 4 Depende de T3
T6 Facturas de servicio	Cabecera (`Factura_del_negocio`) y detalle (`Detalle_de_factura`) guardados juntos con `@Transactional`.	RF-24 a 27 CU-41 a 45	3 de 4 Depende de T5
T7 Ventas de papelería y de ropa	Registrar la venta y descontar el stock. Modificar y eliminar devuelven el stock.	RF-30 a 37 CU-46 a 55	3 de 4 Depende de T6
T8 Ingresos, estadísticas y reportes	Calcular ingresos desde las facturas y ventas, y guardar el historial.	RF-19 CU-24 a 35	4 de 4 Para el final

Orden sugerido: T1, T2, T3 y T4. Después T5, T6, T7 y T8. T1 es la más parecida al molde y deja ver el ciclo completo en un día.

Tareas de documentación

TAREA	QUÉ HAY QUE HACER	ESTADO
D1	Estructura del proyecto con las carpetas nuevas: `backend` con Spring Boot, `postman`, Docker y el CI.	◐ **Borrador listo** PDF «Estructura del proyecto». Revisar, corregir y adoptar.
D2	Versión 1.2 del diagrama de clases en draw.io: capas controller, service y repository, y los DAO que pasan a repositorios.	○ **Pendiente** El esquema de capas del PDF de estructura sirve de guía.
D3	Plan de pruebas para `05-pruebas`: qué cubre `mvn verify`, qué cubre Postman y qué cubre el CI.	◐ **Borrador listo** PDF «Plan de pruebas». Revisar, corregir y adoptar.
D4	Proteger `main` en GitHub (Settings, Branches): exigir pull request y marcar como obligatorias las revisiones «Backend» y «Docker». Solo lo puede hacer quien administra el repositorio.	○ **Pendiente**
D5	Revisar los PR de los demás: leer los cambios, probar la rama y dejar un comentario. Es la mejor forma de entender el código del equipo.	○ **Pendiente**

Cómo pedir ayuda

- Abre el PR como borrador (draft) apenas tengas algo y cuenta dónde te trabaste.
- Si el CI sale en rojo, abre el detalle de la revisión que falló: el error dice el archivo y la línea.
- Una tarea por PR. Más chico es más fácil de revisar.

Lo que sigue por el lado de Osvaldo, propuesto

- El **login real** y los **roles** (administrador y operador), con sesión de 15 minutos (RF-38, RNF-18). T3 sirve de base.
- El **panel interno** en el frontend, módulo por módulo, a medida que las tareas de código dejen listas las rutas.
- Crear el servicio en **Render** y publicar la API.

Decisiones por confirmar

- **El cambio de stack (RNF-17).** La descripción de tecnologías habla de Java con JDBC y DAO. Ahora el backend usa Spring Boot con JPA. RNF-17 pide que cualquier tecnología fuera del rango la apruebe el líder. Falta registrar esa decisión y su aprobación.
- **«Registrarse».** La página tiene ese botón, pero RF-41 dice que el sistema solo tiene dos usuarios internos. ¿Se queda o se quita?
- **Términos y condiciones.** El texto de la página sale de los requisitos y debe aprobarlo la administradora.
- **«Consumir Render».** ¿Alcanza con una API en modo demo con datos de ejemplo, o piden que use la base de datos real?
- **Información destacada (RF-02).** La base no tiene un campo para marcarla. ¿Se agrega o se deja el catálogo completo?

Palabras que verás

PALABRA	QUÉ SIGNIFICA
API, endpoint	Un servicio al que se le piden datos por una dirección, como `/api/servicios`. Cada dirección es un *endpoint*.
JSON	El formato en que la API entrega los datos: una lista de pares nombre y valor.
CRUD	Crear, consultar, actualizar y eliminar.
Swagger	Una página que documenta la API y deja probarla desde el navegador.
CI	Integración continua: GitHub corre las pruebas solo, en cada PR.
CORS	El permiso para que una página en un puerto consulte una API en otro. Sin él, el navegador lo bloquea.
Perfil	El modo en que corre la API: *local* o *demo*.